

GEN AI Developer Assessment

Name: Bhavesh Varma Rudraraju

Date: 30/1/2026

SECTION A: Multiple Choice Questions

Time: 20 minutes | 40 Marks | 2 marks each

Google AI Studio (Q1-Q5)

Q1. In Google AI Studio, what is the primary purpose of "structured prompts"?

a) To generate images from text

Ans->b) **To create reusable prompt templates with examples and variables**

c) To connect to external databases

d) To deploy models to production

Q2. Which model family is available in Google AI Studio for text generation?

a) GPT-4

b) Claude

Ans->c) **Gemini**

d) LLaMA

Q3. What does the "temperature" parameter control in Google AI Studio?

a) Processing speed of the model

Ans->b) **Randomness/creativity of the output**

c) Maximum token limit

d) Model version selection

Q4. In Google AI Studio, what is a "freeform prompt"?

a) A prompt with predefined structure and examples

Ans->b) **An open-ended prompt without structured format**

c) A prompt that only accepts JSON input

d) A prompt connected to external APIs

Q5. What file format does Google AI Studio use when you export a prompt to code?

a) .txt only

Ans->b) **.py (Python) and other language options**

c) .exe

d) .prompt

Python FastAPI (Q6-Q10)

Q6. What is the correct way to define a POST endpoint in FastAPI?

- a) `@app.post("/items/")`
- b) `@app.POST("/items/")`
- c) `@post.app("/items/")`

Ans->d) `@route.post("/items/")`

Q7. Which library does FastAPI use for data validation?

- a) Marshmallow
- b) Cerberus

Ans->c) `Pydantic`

- d) Django REST Framework

Q8. What command runs a FastAPI application using Uvicorn?

- a) `python app.py`
- b) `fastapi run app.py`

Ans->c) `uvicorn main:app --reload`

- d) `flask run`

Q9. In FastAPI, how do you define an optional query parameter?

- a) `def read_item(q: str):`

Ans->b) `def read_item(q: str = None):`

- c) `def read_item(q: Optional[str]):`

- d) Both b and c are correct

Q10. What does FastAPI automatically generate at `/docs`?

- a) Database documentation

Ans->b) `Interactive Swagger UI API documentation`

- c) Source code

- d) Error logs

LangChain (Q11-Q15)

Q11. What is the primary purpose of a "Chain" in LangChain?

a) To store data in a database

Ans->b) **To sequence multiple LLM calls or operations together**

c) To encrypt API keys

d) To manage user authentication

Q12. Which component in LangChain is used to split large documents into smaller chunks?

a) DocumentLoader

Ans->b) **TextSplitter**

c) VectorStore

d) Embedder

Q13. What is the role of RetrievalQA chain in LangChain?

a) To train new models

Ans->b) **To retrieve relevant documents and answer questions based on them**

c) To translate languages

d) To generate images

Q14. In LangChain, what does an "Embedding" represent?

a) A compressed version of the original text

Ans->b) **A numerical vector representation of text**

c) An encrypted form of data

d) A summary of the document

Q15. Which is a valid way to initialize ChatGoogleGenerativeAI in LangChain?

Ans->a) **ChatGoogleGenerativeAI(model="gemini-pro")**

b) GoogleChat(model="gemini")

c) GeminiLLM(version="pro")

d) LangChainGoogle(model="gemini")

LangGraph (Q16-Q20)

Q16. What is LangGraph primarily used for?

a) Creating static flowcharts

Ans->b) **Building stateful, multi-actor applications with LLMs**

c) Visualizing training data

d) Database schema design

Q17. In LangGraph, what is a "Node"?

a) A database connection

Ans->b) **A function that performs a specific task in the graph**

c) A user interface element

d) An API endpoint

Q18. What does the StateGraph class in LangGraph manage?

a) User sessions only

Ans->b) **The flow and state of the application across nodes**

c) Database transactions

d) File storage

Q19. How do you define conditional routing in LangGraph?

a) Using if-else in the node function only

Ans->b) **Using add_conditional_edges() method**

c) Using switch statements

d) Conditional routing is not supported

Q20. What is the purpose of END in LangGraph?

a) To terminate the Python program

Ans->b) **To mark the final node where graph execution stops**

c) To close database connections

d) To log errors

SECTION B: Case Study — Build a RAG Chatbot

SCENARIO: Customer Support Chatbot for TechGear Electronics

TechGear Electronics needs a customer support chatbot that answers product questions using a knowledge base. Build a RAG (Retrieval Augmented Generation) chatbot using ChromaDB, LangChain, and LangGraph.

Knowledge Base (product_info.txt)

Product: SmartWatch Pro X Price: ₹15,999 | Features: Heart rate, GPS, 7-day battery, water resistant 50m Warranty: 1 year standard, 2 years extended (₹1,999) Product: Wireless Earbuds Elite Price: ₹4,999 | Features: ANC, 24-hour battery, Bluetooth 5.2 | Warranty: 6 months Product: Power Bank Ultra 20000mAh Price: ₹2,499 | Features: Fast charging 22.5W, USB-C & USB-A | Warranty: 1 year Return Policy: 7-day no-questions-asked. Refund in 5-7 business days. Support: Mon-Sat, 9AM-6PM IST | support@techgear.com

Tasks

Task 1: Setup & Document Loading [10 marks] → Load the knowledge base text file → Split into chunks using LangChain's TextSplitter → Create embeddings and store in ChromaDB

Task 2: RAG Chain Implementation [20 marks] → Create a retriever from ChromaDB → Build a RAG chain that retrieves context and generates answers → Use Google's Gemini model via LangChain

Task 3: LangGraph Workflow [20 marks]

→ Node 1: Classifier — Categorize query (products / returns / general)

→ Node 2: RAG Responder — Use RAG to answer

→ Node 3: Escalation — Return escalation message for unhandled queries → Implement conditional routing based on classifier output

Task 4: FastAPI Endpoint [10 marks]

→ Create POST endpoint accepting user query

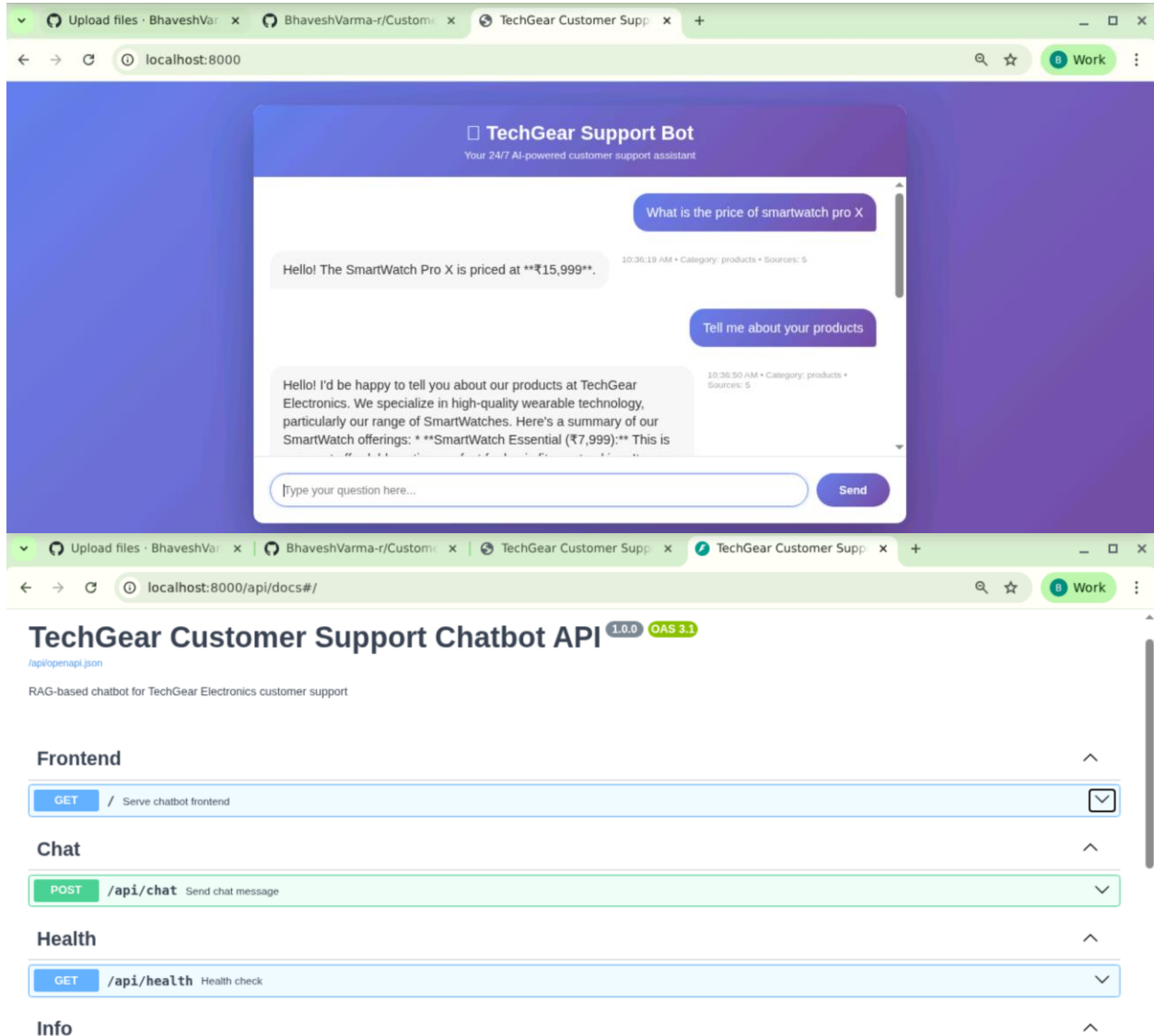
→ Process through LangGraph workflow

→ Return chatbot response as JSON

Solution:

Github Repo Link: https://github.com/BhavesVarma-r/Customer_SupportBot

Screenshots:



Frontend

GET / Serve chatbot frontend

Chat

POST /api/chat Send chat message

Send a message to the chatbot and receive a response.

The chatbot processes the query through a multi-stage workflow:

1. **Classification:** Categorizes the query (products/returns/warranty/support/general)
2. **Retrieval:** Retrieves relevant documents from knowledge base using ChromaDB
3. **Generation:** Generates answer using RAG with Google Gemini
4. **Routing:** Routes to appropriate handler or escalation if needed

Query will be stored in interactions database for analytics.

Args: message: Chat message containing query and optional conversation_id

Returns: ChatResponse with answer and metadata

Raises: HTTPException: If processing fails

Parameters

Try it out